Data Structures (CS2103)

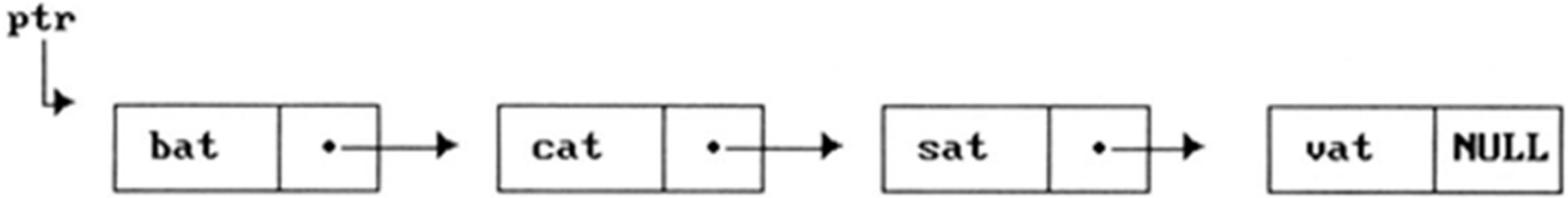
Linked Lists

Prepared By – Ashish K Layek

CST, IIEST Shibpur

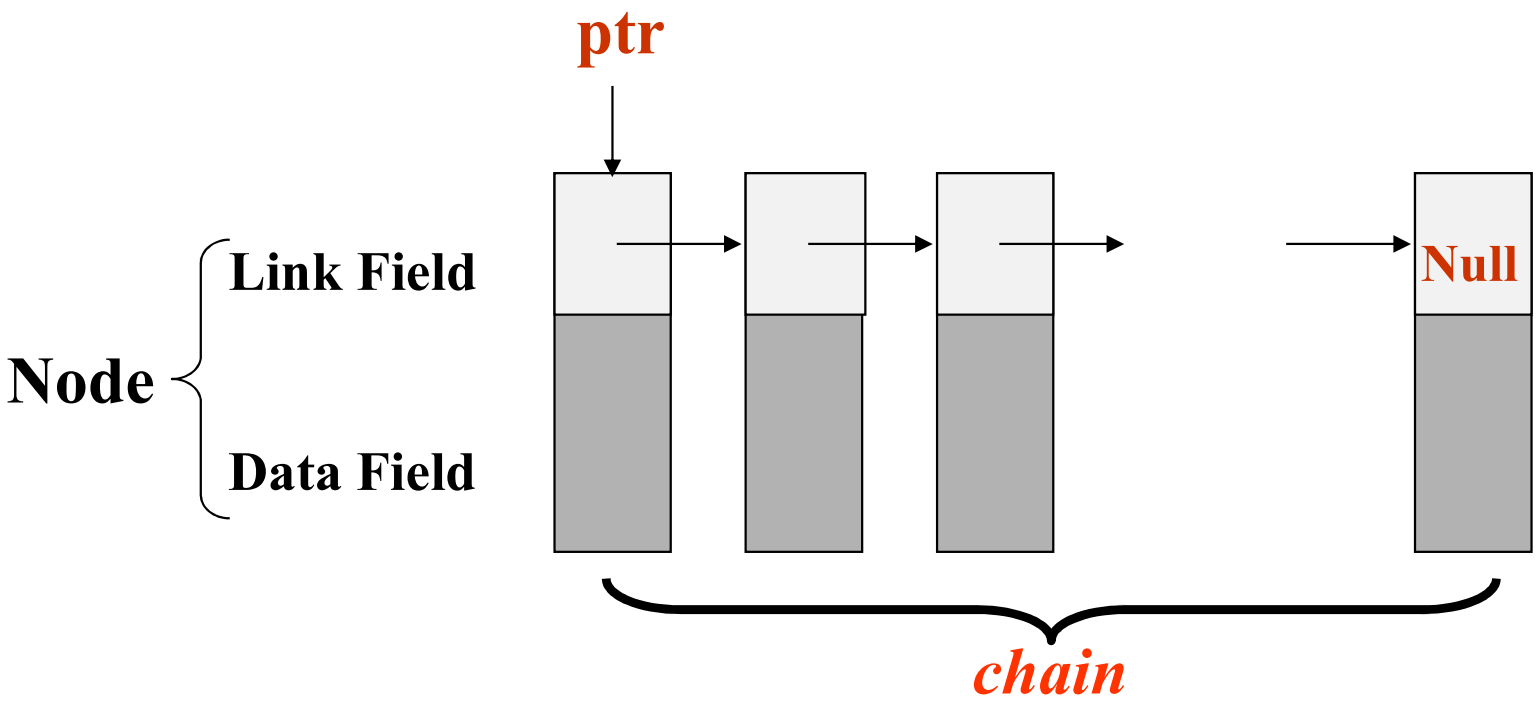
Singly Linked Lists

- Linked lists are drawn as an order sequence of nodes with links represented as arrows
  - The name of the pointer to the first node in the list is the name of the list. (called *ptr*.)
    - Notice that we do not explicitly put in the values of pointers, but simply draw allows to indicate that they are there.



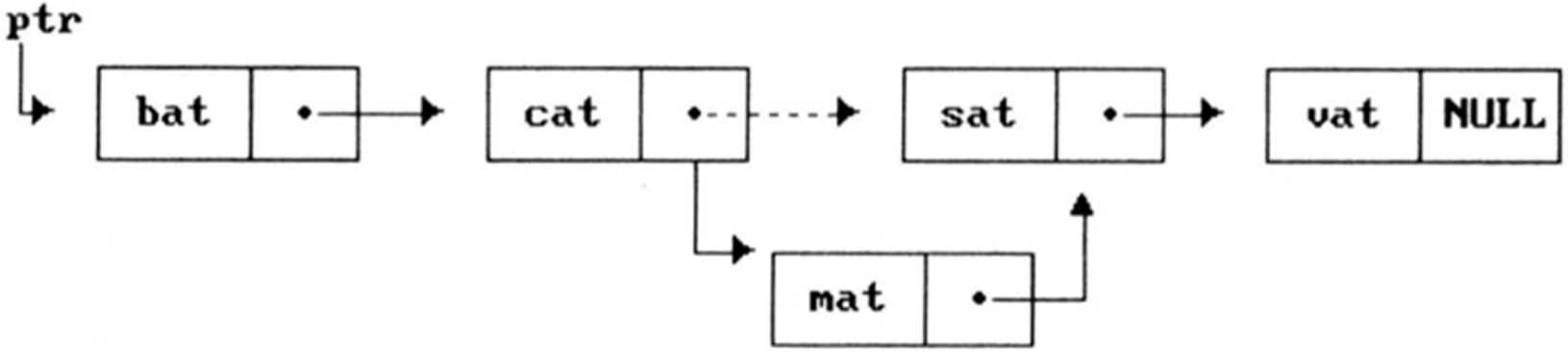
Singly Linked Lists Contd…

- The nodes do not resident in sequential locations
- The locations of the nodes may change on different runs



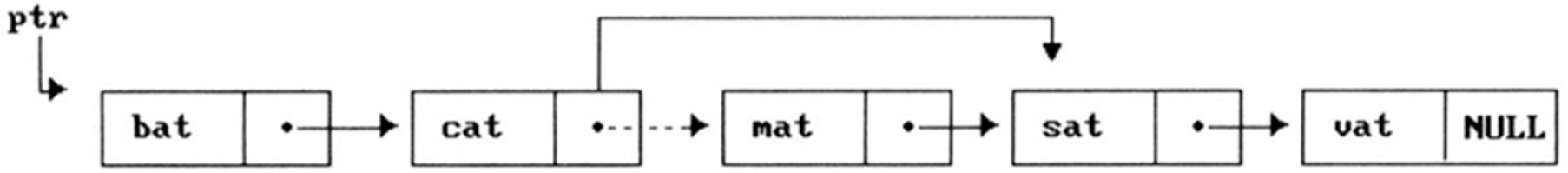
Singly Linked Lists: Easier to insert

- Why it is easier to make arbitrary insertions and deletions using a linked list?
  - To insert the word *mat* between *cat* can *sat*, we must:
    - ✓ Get a node that is currently unused; let its address be *paddr*.
    - ✓ Set the data field of this node to *mat*.
    - ✓ Set *paddr*'s link field to point to the address found in the link field of the node containing *cat*.
    - ✓ Set the link field of the node containing *cat* to point to *paddr*.



Singly Linked Lists: Easier to delete

- Delete *mat* from the list:
  - ✓ We only need to find the element that immediately precedes *mat*, which is *cat*, and set its link field to point to *mat's* link
  - ✓ We have not moved any data, and although the link field of mat still points to sat, *mat* is no longer in the list.



Singly Linked Lists Contd…

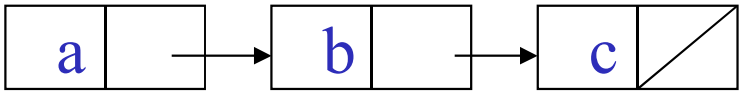
- We need the following capabilities to make linked representations possible:
  - ✓ Defining a node's structure, that is, the fields it contains. We use *self-referential structures*
  - ✓ Create new nodes when we need them. (*malloc*)
  - ✓ Remove nodes that we no longer need. (*free*)

Singly Linked Lists: Representation

- Self-Referential Structures
  - One or more of its components is a pointer to itself.

```
typedef struct list {
    char data;
    struct list *next;
} list;
```

Construct a list with three nodes




```
list item1, item2, item3;
item1.next = item2.next = item3.next = NULL;
item1.data ='a';
item2.data ='b';
item3.data ='c';
item1.next = &item2;
item2.next = &item3;
```

- The list is created using stack memory. Therefore, the lifespan of the list is until that function is in execution. Once the function call is over, list becomes invalid. So this is not a suitable approach.
- So, use dynamic memory allocation to create the list from heap memory, that will be persistent across function calls.
  - ✓ malloc: obtain a node
  - ✓ free: release memory

Singly Linked Lists: Representation

- Self-Referential Structures

```
typedef struct list {
    int data;
    struct list *next;
} list;
```

- Creation

```
list * ptr = NULL;
```

- Allocation

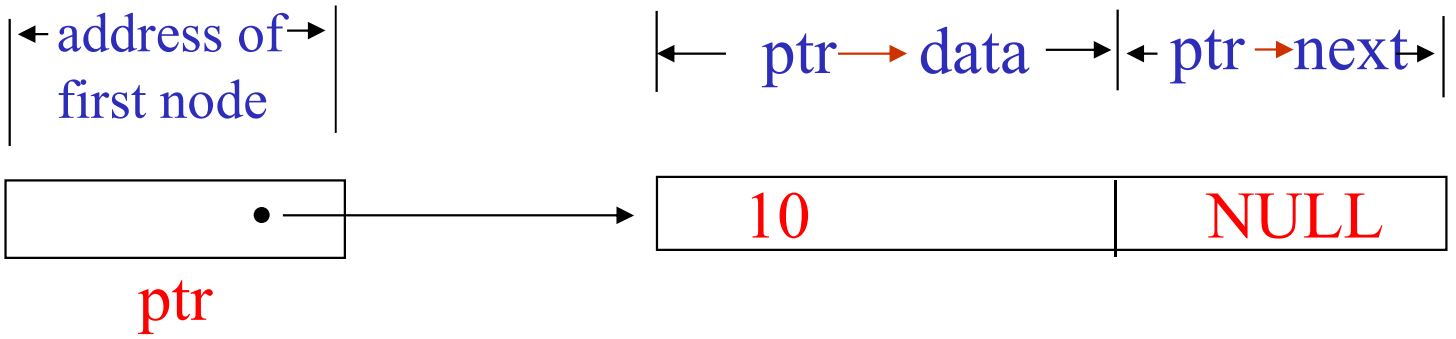
```
ptr =(list *) malloc
   (sizeof(list));
```

- Data

```
ptr -> data = 10;
ptr -> next = NULL;
```

- Return the spaces

```
free(ptr);
```



Singly Linked Lists: Example

- Example: *List of words*

  - Declaration

```
typedef struct list_node {
     char data [4];
     struct list_node * next;
} list_node;
```

  - Creation

```
list_node * ptr = NULL;
```

  - Allocation

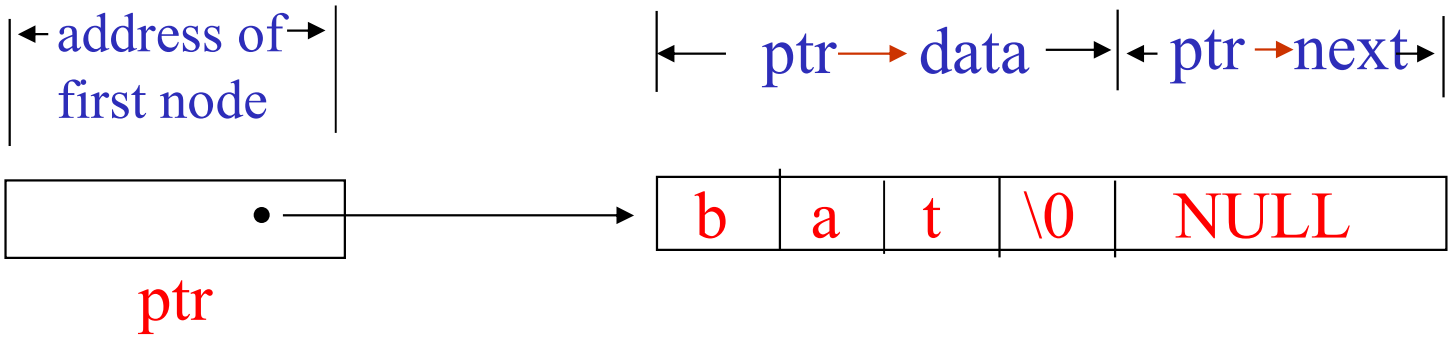
```
ptr =(list_node *) malloc
   (sizeof(list_node));
```

  - Data

```
strcpy(ptr -> data, "bat");
ptr -> next = NULL;
```

  - Return the spaces

```
free(ptr);
```



Singly Linked Lists: Using Heap Memory

Example: *Two-node linked list*

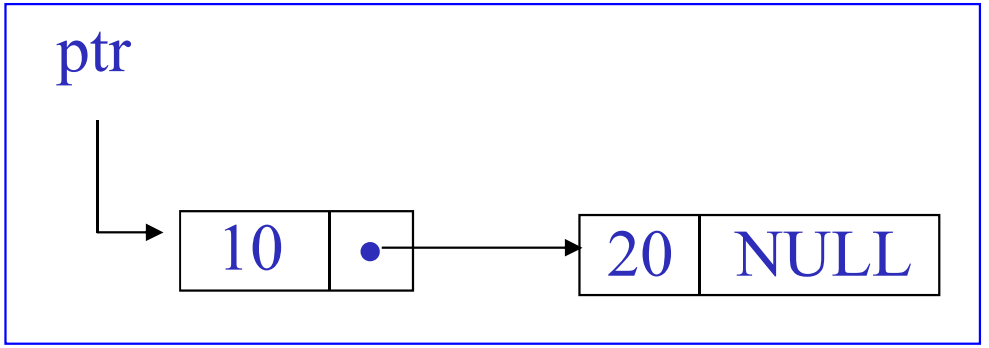
```
typedef struct list_node {
   int data;
   struct list_node * next;
} list_node;
```

Program

```
void main() {
    list_node * ptr = NULL;
    ptr = create2();

    //Also need to free the
      Linked List -- ??

}
```

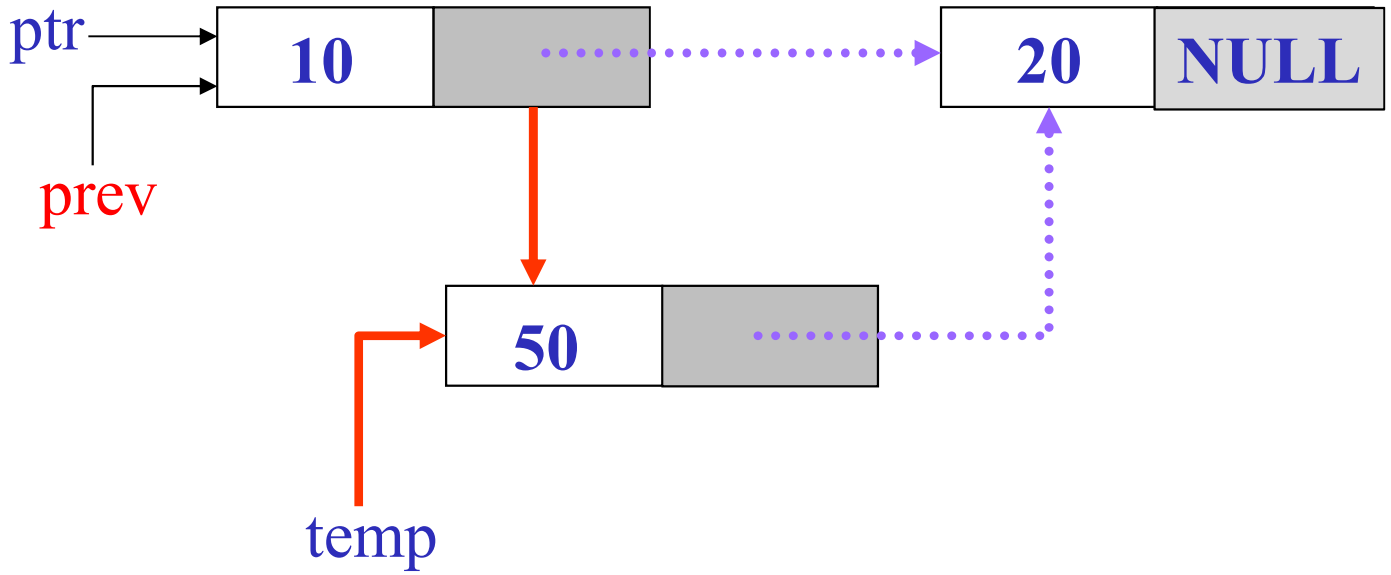
Function:

```
list_node * create2() {
   /* create a linked list with two nodes */
   list_node * first = NULL;
   list_node * second = NULL;
   first = (list_node *) malloc(sizeof(list_node));
   second = (list_node *) malloc(sizeof(list_node));
   first->next = NULL;
   first->data = 10;
    second->next = NULL;
   second->data = 20;
   first->next = second;
   return first;
}
```



Singly Linked Lists: Insertion

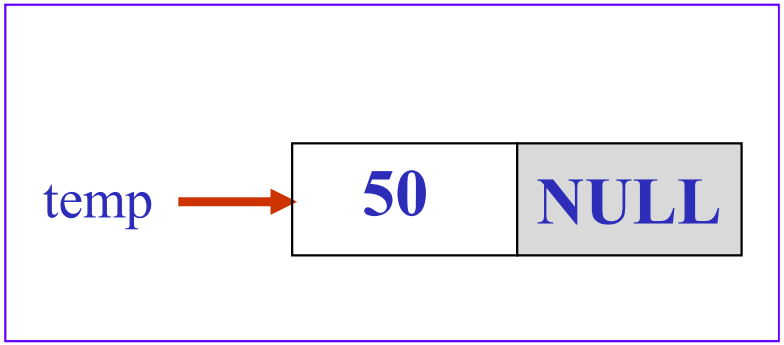
- Insertion: insert a new node with data = 50 into the list ptr after some arbitrary node in the list prev



Singly Linked Lists: Insertion Contd…

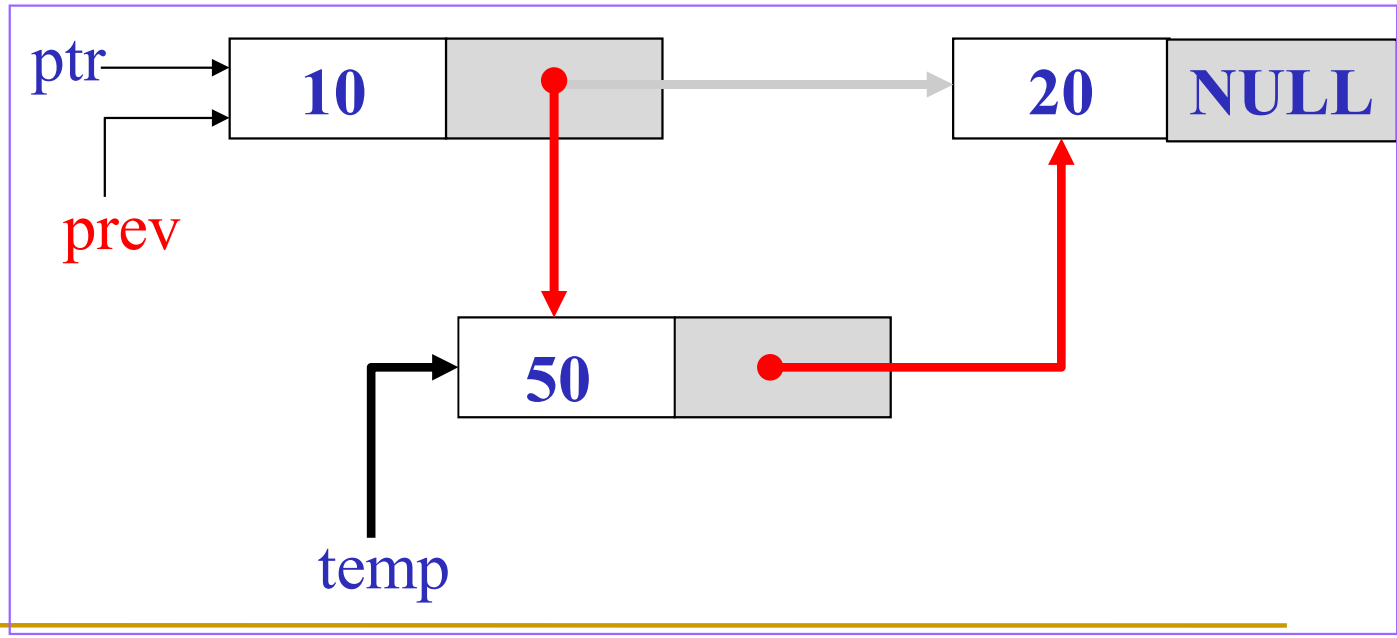
- Implement Insertion:

```
list_node * insert(list_node * ptr, list_node * prev, int val) {
  /* insert a new node with data = val into the list ptr after
  some arbitrary node in the list prev */

  list_node * temp = NULL;
  temp = (list_node *)malloc(sizeof(list_node));
  if( temp == NULL){
      printf("The memory is full\n");
      exit(1);
  }
  temp->data = val; // update data
  temp->next = NULL

  // continued in next slide
```



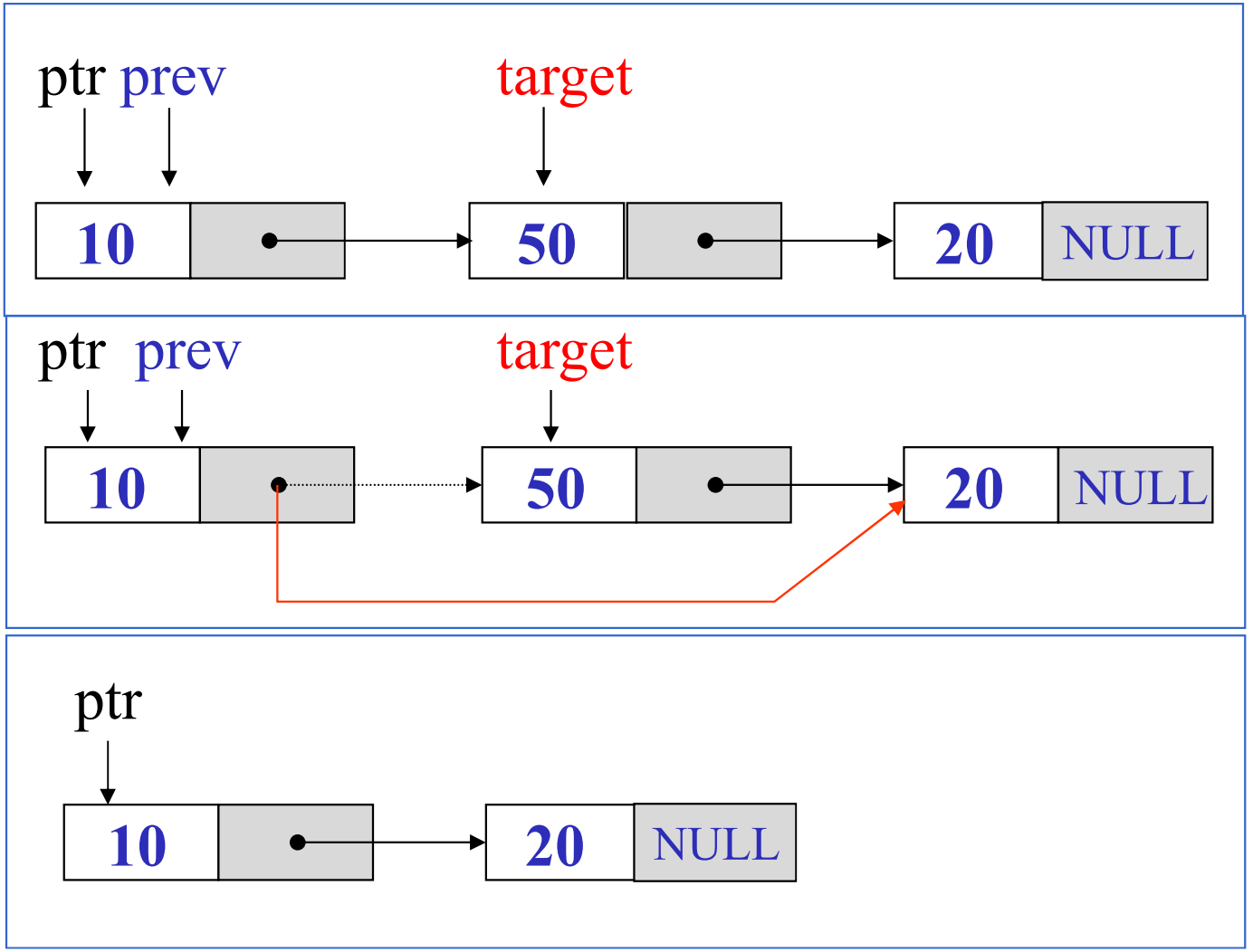
Singly Linked Lists: Insertion Contd…

```
    // Incase prev is null, then add at beginning of the list
    if (prev == NULL) {
         temp->next = ptr;
         ptr = temp;
         return ptr;
    }
    if(ptr){ //nonempty list
         temp->next = prev->next;
         prev->next = temp;
    }
    else { //empty list
         ptr = temp;
    }
    return ptr;
}
```



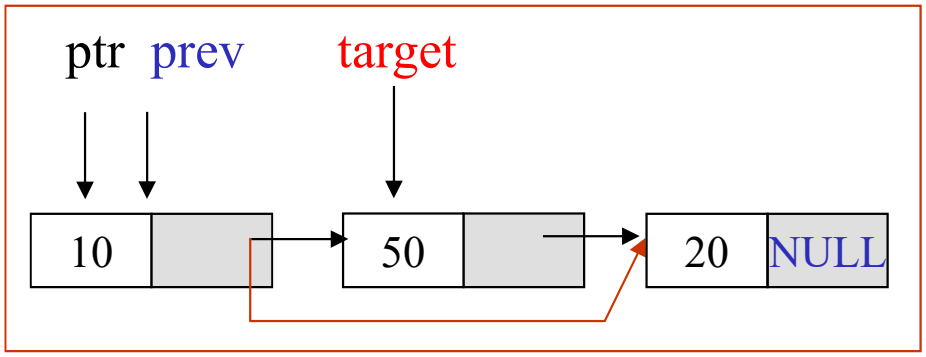
Singly Linked Lists: Deletion

- Deletion: delete a node from the list ptr.
- Let's call it target, some arbitrary node in the list.



Singly Linked Lists: Deletion Contd…

```
list_node * delete(list_node * ptr, list_node * prev) {
    /* Delete node target from the list, and prev be the node preceding
    to target. The prev is NULL if target is the first node in list */

    list_node * target = NULL;
    if(prev) { // target is not the first node
        target = prev->next;
        prev->next = target->next;
    }
    else{ // target is the first node
       target = ptr;
       ptr = ptr->next;
    }
    free(target);
    return ptr;
}
```



Singly Linked Lists: Display

- Print out a list (traverse a list)

```
void print_list(list_node *ptr)
{
    printf("The list: ");
    while(ptr != NULL){
        printf("%d -> ", ptr->data);
        ptr = ptr->next;
    }
    printf("NULL\n\n");
}
```

Singly Linked Lists: Demonstration of Insertion and Deletion

```
void main() {
    list_node * list = NULL;

    list = insert(NULL, NULL, 10);
    print_list(list);
    list = insert(list, NULL, 20);
    print_list(list);
    list = insert(list, list->next, 30);
    print_list(list);

    list = delete(list, list->next);
    print_list(list);
    list = delete(list, list);
    print_list(list);
    list = delete(list, NULL);
    print_list(list);
}
```

Output

```
The list: NULL

The list: 10 -> NULL

The list: 20 -> 10 -> NULL

The list: 20 -> 10 -> 30 -> NULL

The list: 20 -> 10 -> NULL

The list: 20 -> NULL

The list: NULL
```

Singly Linked Lists: Insertion Contd…

- But how to find the preceding node prev after which the new node will be inserted?
  - This is determined based on the problem statement.
  - Example: Lets assume a list stores integers in ascending order of their values. You need to insert an element with value as 50.
    - ✓ In that case you need to search the list from beginning and find the previous node than a node of list which has value greater than or equal to 50.

```
list_node * findPrevNodeForInsertAscending(
                    list_node *ptr, int val) {

    /* Empty list case */
    if(ptr == NULL)
        return NULL;

    /* check if the new node to be added first */
    if(val < ptr->data ) {
        /* to be added first  */
        return NULL;
    }

    list_node * prev = NULL;

    while((ptr != NULL) && (val > ptr->data)){
        prev = ptr;
        ptr = ptr->next;
    }

    return prev;
}
```

Singly Linked Lists: Insertion Contd…

- In practice, we will not write insert methods in this two step approach (as explained in previous slide). This was only for explaining purpose.
- We will write a single method according to the problem statement as stated below.

```
list_node * insertNodeAscending(
                  list_node *ptr, int val){

    /* Create a tem node first */
    list_node * temp = NULL;
    temp = (list_node*)malloc(
                        sizeof(list_node));
    if( temp == NULL){
        printf("The memory is full\n");
        exit(1);
    }
    temp->data = val; // Update data
    temp->next = NULL; // Set to null


    /* Empty list case, the temp node
     itself is the list */
    if(ptr == NULL) {
        return temp;
    }

---->
```

```
    /* Hold the list head */
    list_node * head = ptr;

    /* check if the new node to be added first */
    if(val < ptr->data ) {
        /* Add at first */
        temp->next = ptr;
        return temp;
    }

    /* New node cannot be added as first node */
    list_node * prev = NULL;
    while((ptr != NULL) && (val > ptr->data)){
        prev = ptr;
        ptr = ptr->next;
    }

    temp->next = prev->next;
    prev->next = temp;

    return head;
}
```

Singly Linked Lists: Deletion Contd…

- And also how to find the preceding node prev after which the target node will be deleted?
  - This is again determined based on the problem statement.
  - Example: Lets assume a list stores integers in random order of their values. You need to delete the first occurrence of node having value as 50.
    - ✓ In that case you need to search the list from beginning and find the previous node than a node of list which has value equal to 50.

```
list_node * findPrevNodeForDelete(
        list_node *ptr, int key, ...) {

    Do it yourself

}
```

Singly Linked Lists: Deletion Contd…

- Similarly, we will not write delete methods in this two step approach (as explained in previous slide). This was only for explaining purpose.
- We will write a single method according to the problem statement as stated below.

```
list_node * deleteNodeUnordered(
                    list_node *ptr, int val){

    printf("Element to be deleted: %d \n", val);

    /* Empty list case */
    if(ptr == NULL) {
        printf("Empty list.\n");
        printf("Element %d not found\n", val);
        return NULL;
    }

    /* Hold the list head */
    list_node * head = ptr;

    /*check if the first node is target node */
    if(ptr->data == val) {
        /* Found the element */
        head = head->next;
        free(ptr);
        return head;
    }
```

```
    /* First node is not the target node */
    list_node * prev = NULL;

    while((ptr != NULL) && (ptr->data != val)){
        prev = ptr;
        ptr = ptr->next;
    }

    if(ptr == NULL) {
        /* Element not found case */
        printf("Element  %d not found\n", val);
    }
    else {
        /* Element found case */
        prev->next = ptr->next;
        free(ptr);
    }

    return head;
}
```

Singly Linked Lists: Demonstration 2 of Insertion and Deletion

- Problem Statement: Insert a random set of elements in the list according to the ascending order and perform deletion without the knowledge of the order of the elements.

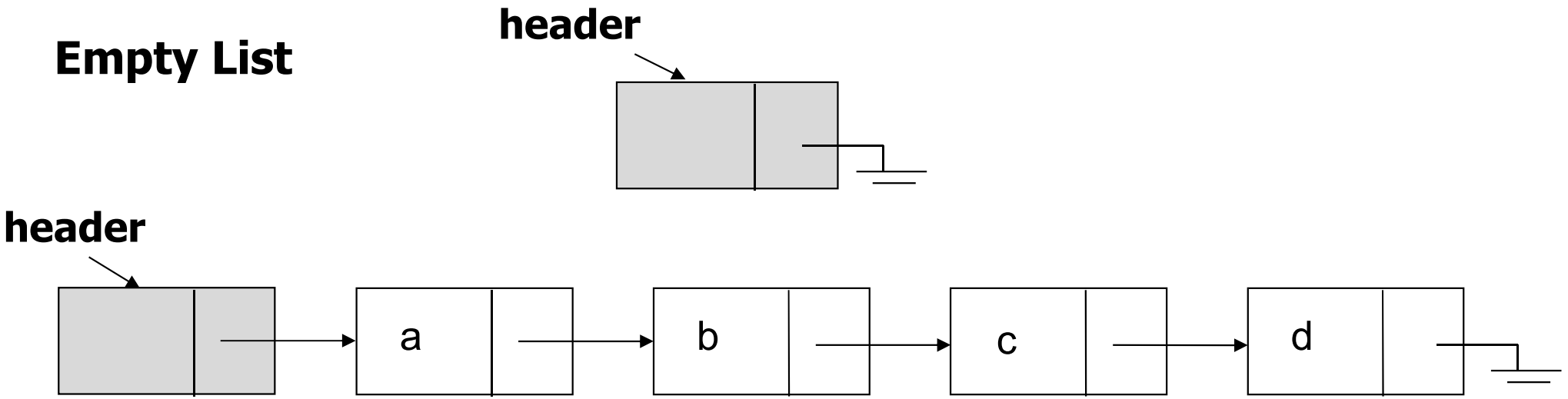
```
void main() {

    list_node * list = NULL;
    list = insertNodeAscending(NULL, 10);
    print_list(list);
    list = insertNodeAscending(list, 0);
    print_list(list);
    list = insertNodeAscending(list, -1);
    print_list(list);
    list = insertNodeAscending(list, 15);
    print_list(list);
    list = deleteNodeUnordered(list, 15);
    print_list(list);
    list = deleteNodeUnordered(list, 0);
    print_list(list);
    list = deleteNodeUnordered(list, 5);
    print_list(list);
    list = deleteNodeUnordered(list, 10);
    print_list(list);

}
```

Output

```
The list:10 -> NULL

The list:0 -> 10 -> NULL

The list:-1 -> 0 -> 10 -> NULL

The list:-1 -> 0 -> 10 -> 15 -> NULL

The list:-1 -> 0 -> 10 -> NULL

The list:-1 -> 10 -> NULL

Element with value: 5 not found
The list:-1 -> 10 -> NULL

The list:-1 -> NULL
```

Singly Linked Lists with Header Nodes

- One problem with the basic description: it assumes that whenever an item x is removed (or inserted) some previous item is always present.
- Consequently removal of the first item and inserting an item as a new first node become special cases to consider.
- In order to avoid dealing with special cases: introduce a **header node (dummy node)**. *This is not a mandatory item, it just makes thing easy.*
- A header node is an extra node in the list that holds no data but serves to satisfy the requirement that every node has a previous node.



Representing Large number using Singly Linked List

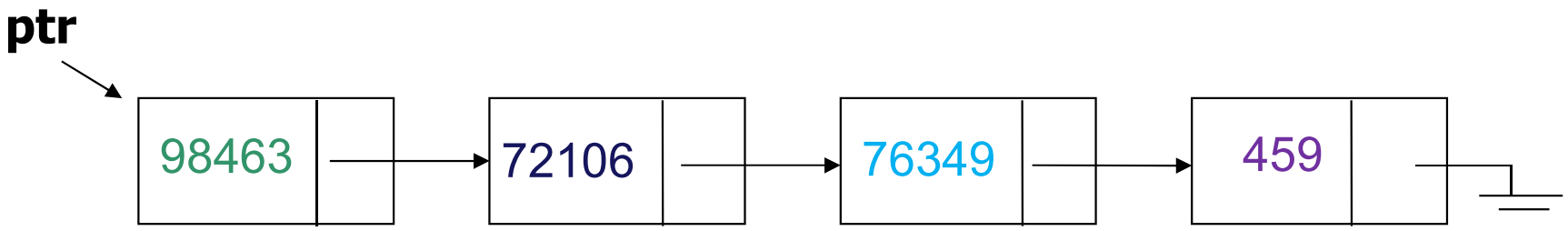
- The hardware of most computers allows integers of only a specific maximum length.
- Suppose that we wish to represent positive integers of arbitrary length, linked list could be a good choice.
- This suggests representing long integers by storing their digits from right to left in a list.
- However, to save space, we keep only a few digits in each node
  - For example, we may keep 5 digits in each node (numbers as large as 99999).

```
struct largeNumNode {
    int digits;
    struct largeNumNode * next;
};
```

This data type may change depending upon how many digits you want to store in a node.

- ✓ char for 1/2 digit(s)
- ✓ short for 3/4 digits
- ✓ int for 5/9 digits

For example, the integer 45976349721069 8463 is represented by the list illustrated in the Figure

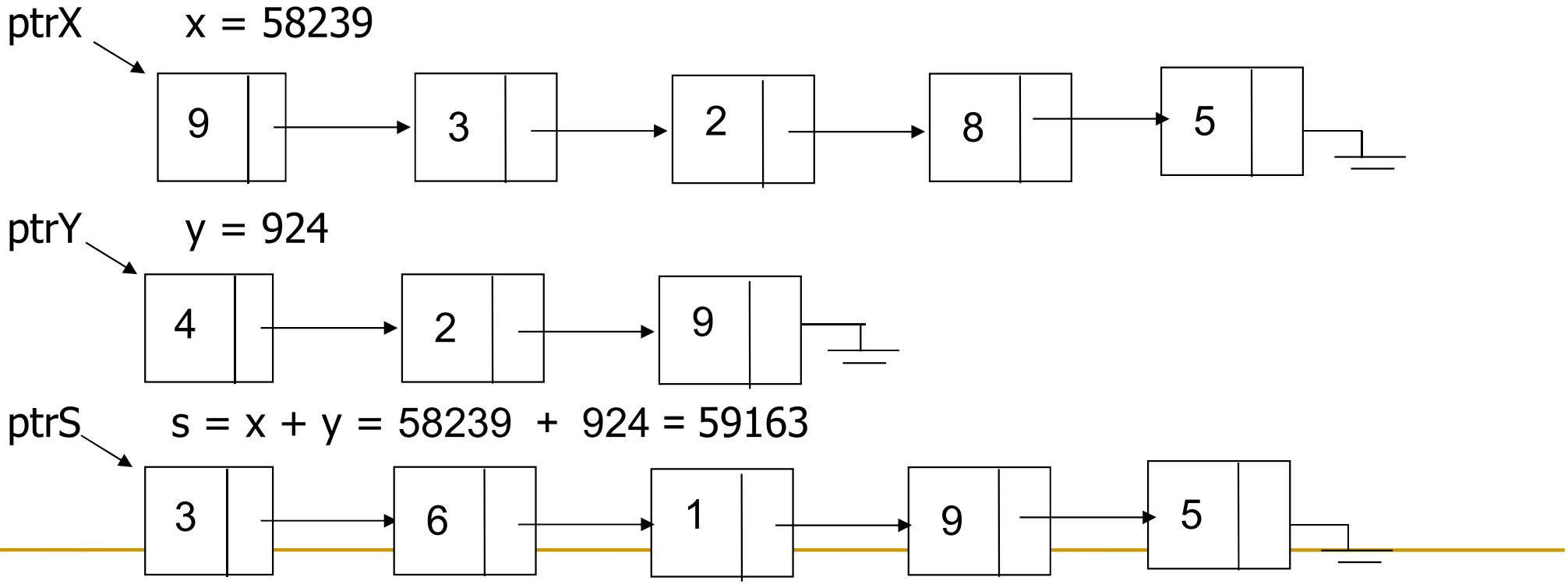


Addition of Large number

- For sake of better understanding, lets store single digit in respective node of the linked list.
- The function to add two long number accepts pointers to two such lists representing integers, creates a list representing the sum of the integers, and returns a pointer to the sum list.
- Both lists are traversed in parallel, and single digit is added at a time.
- If the sum of two digit numbers is x, the low-order digit of x can be extracted by using the expression x % 10, which yields the remainder of x on division by 10.
- The carry can be computed by the integer division x/l0.
- When the end of one list is reached, the carry is propagated to the remaining digits of the other list.

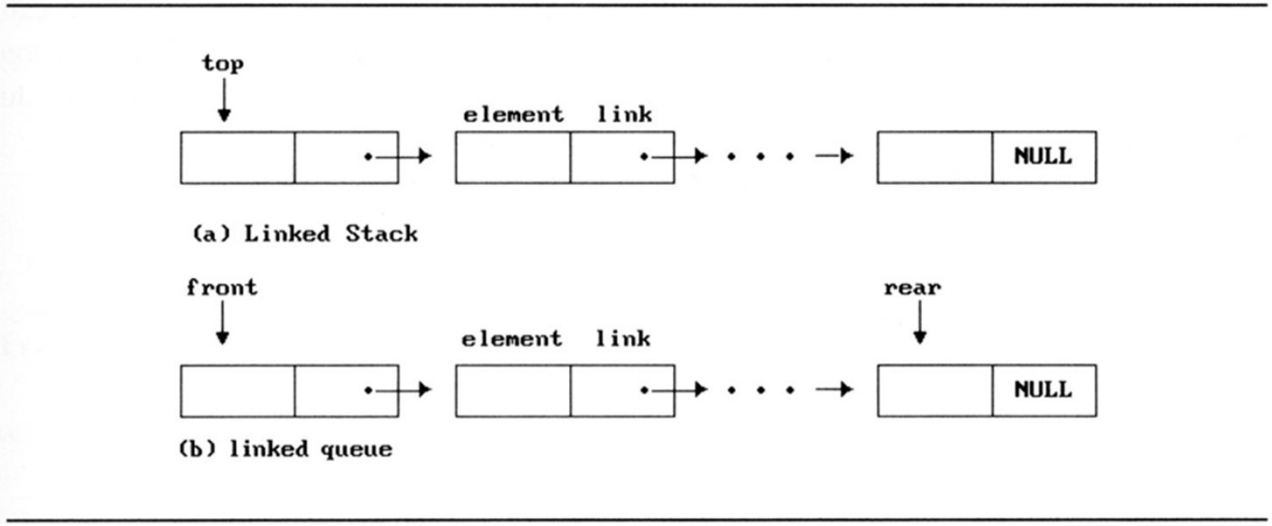
```
struct largeNumNode {
   char digit;
   struct largeNumNode * next;
};
```

Code addLargeNumbers(…) yourself



Dynamically Linked Node Stacks and Queues

- Notice that direction of links for both stack and the queue facilitate easy insertion and deletion of nodes.
  - Easily add or delete a node form the top of the stack.
  - Easily add a node to the rear of the queue and add or delete a node at the front of a queue.
  - Operations like push(), pop(), enqueue(), dequeue() can be achieved in O(1)



Linked stack and queue

Representing Polynomials As Singly Linked Lists

- The manipulation of symbolic polynomials, has a classic example of list processing.
- In general, we want to represent the polynomial:

$$A(x) = a_{m-1}x^{e_{m-1}} + \cdots + a_0 x^{e_0}$$

  - Where the a_i are nonzero coefficients and the e_i are nonnegative integer exponents such that

    $e_{m-1} > e_{m-2} > \ldots > e_1 > e_0 \geqq 0$.

  - We will represent each term as a node containing coefficient and exponent fields, as well as a pointer to the next term.

Representation of Polynomials

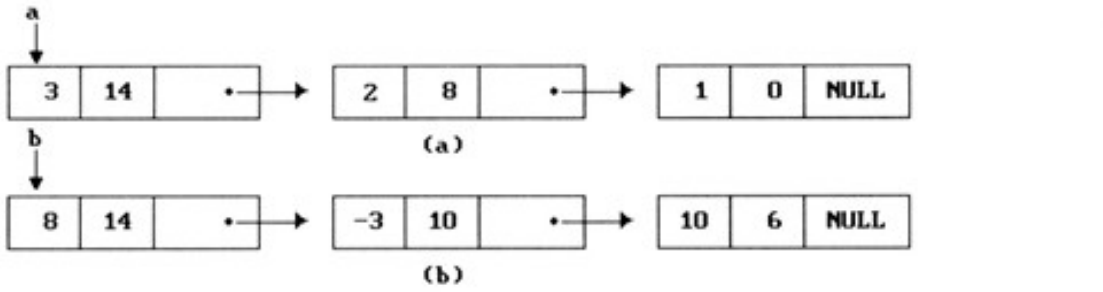
- Assuming that the coefficients are integers, the type declarations are:

```
struct poly_node {
    int coef;
    int expon;
    struct poly_node * next;
};
```

coef	expon	next

$$a = 3x^{14} + 2x^{8} + 1$$

$$b = 8x^{14} - 3x^{10} + 10x^{6}$$



Polynomial representation

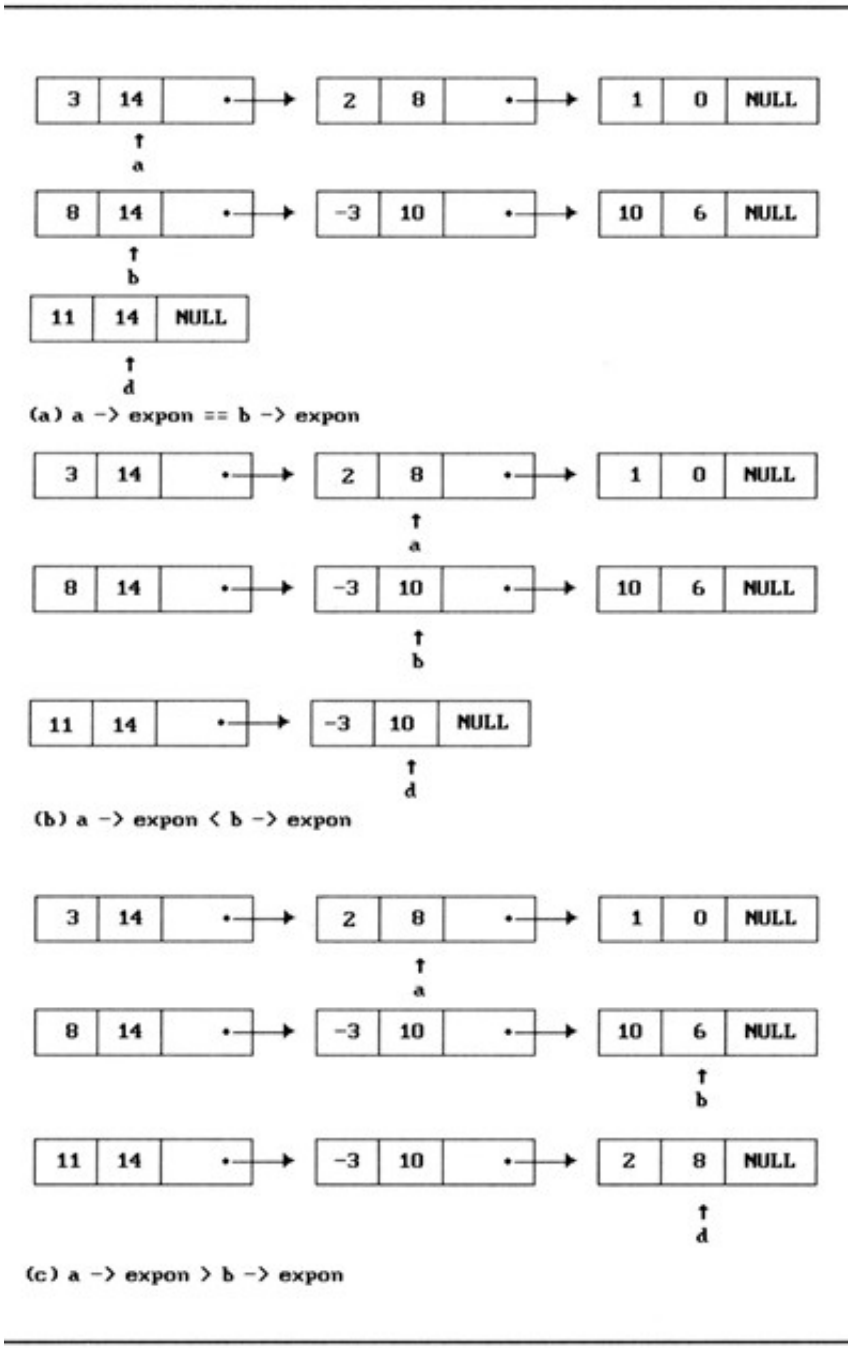
Adding Polynomials

- To add two polynomials, we examine their terms starting at the nodes pointed to by *a* and *b*.
  - If the exponents of the two terms are equal
    1. add the two coefficients
    2. create a new term for the result.
  - If the exponent of the current term in *a* is less than *b*
    1. create a duplicate term of *b*
    2. attach this term to the result, called *d*
    3. advance the pointer to the next term in *b*.
  - We take a similar action on *a* if *a->expon > b->expon.*

Adding Polynomials

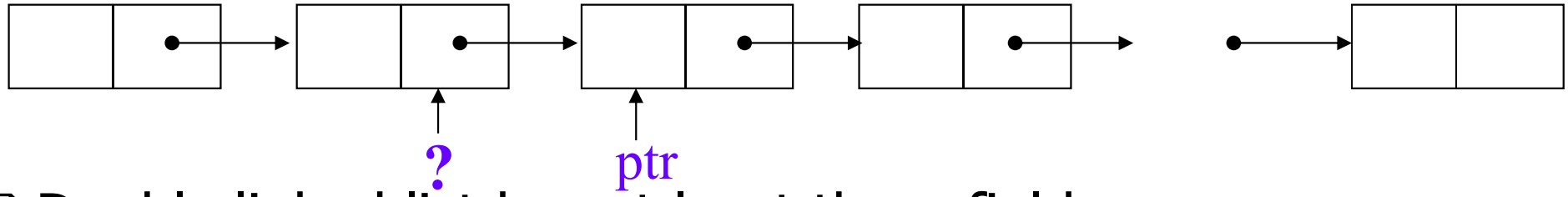
Pictorial description of generating the first three term of $d = a+b$

Code: Do it yourself



Doubly Linked Lists

- Singly linked lists pose problems because we can move easily only in the direction of the links



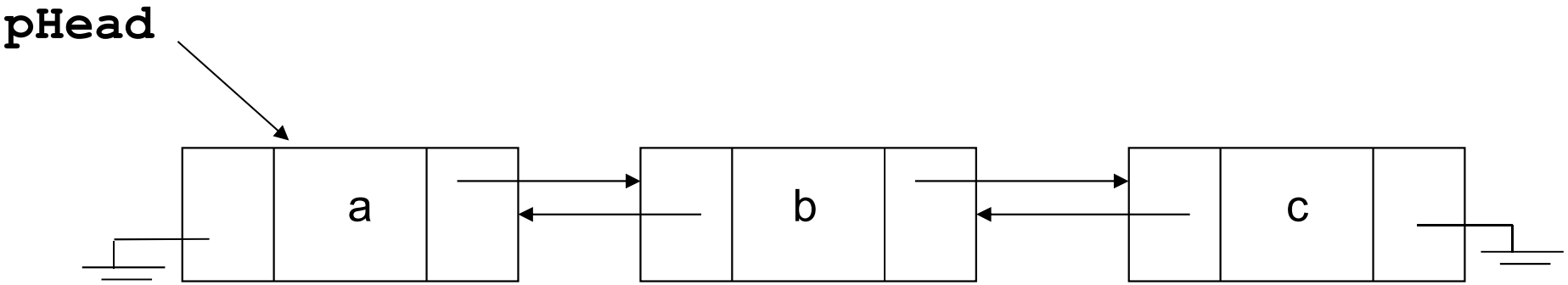
- Doubly linked list has at least three fields

  left link field(*prev*), data field, right link field(next).

  - The necessary declarations:

```
struct dll_node {
    struct dll_node * prev;
    int data;
    struct dll_node * next;
};
```

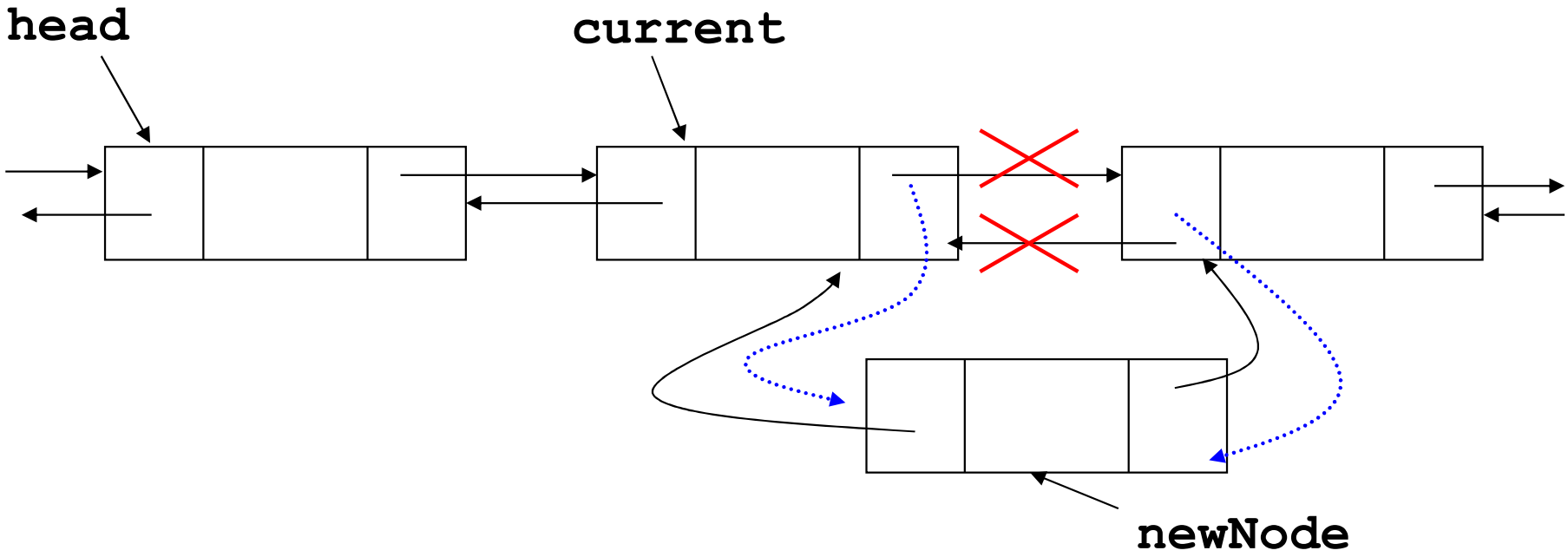
Doubly Linked Lists



- **Advantages:**
  - Convenient to traverse the list backwards.
  - Simplifies insertion and deletion because you no longer have to refer to the previous node.
- **Disadvantage:**
  - Increase in space requirements.

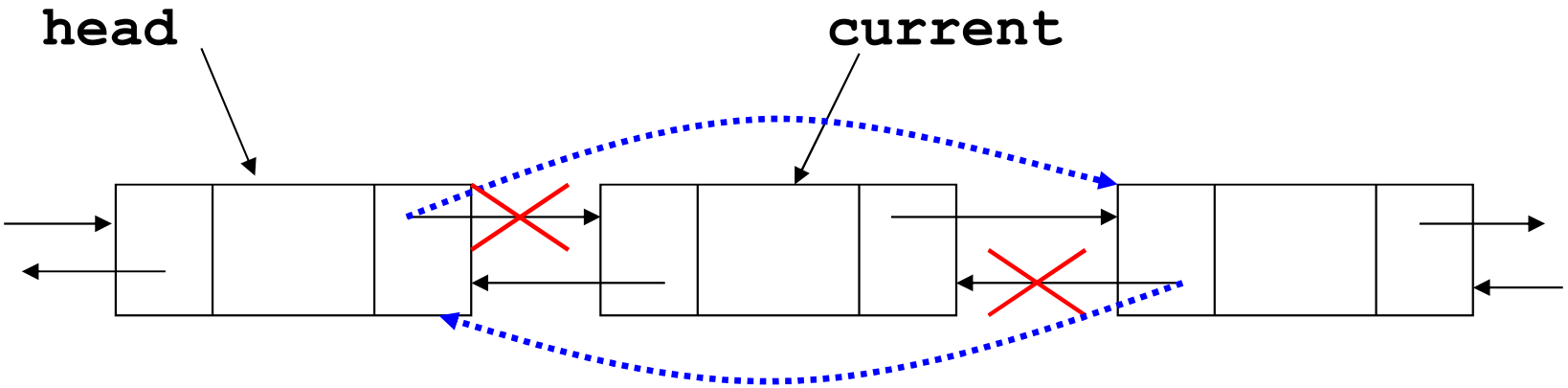
```
struct dll_node {
    int data;
    struct dll_node * prev;
    struct dll_node * next;
};
```

Doubly Linked Lists: Insertion




```
newNode = (struct dll_node *)malloc(sizeof(struct dll_node));
newNode->prev = current;
newNode->next = current->next;
newNode->prev->next = newNode;
newNode->next->prev = newNode;
current = newNode;
```

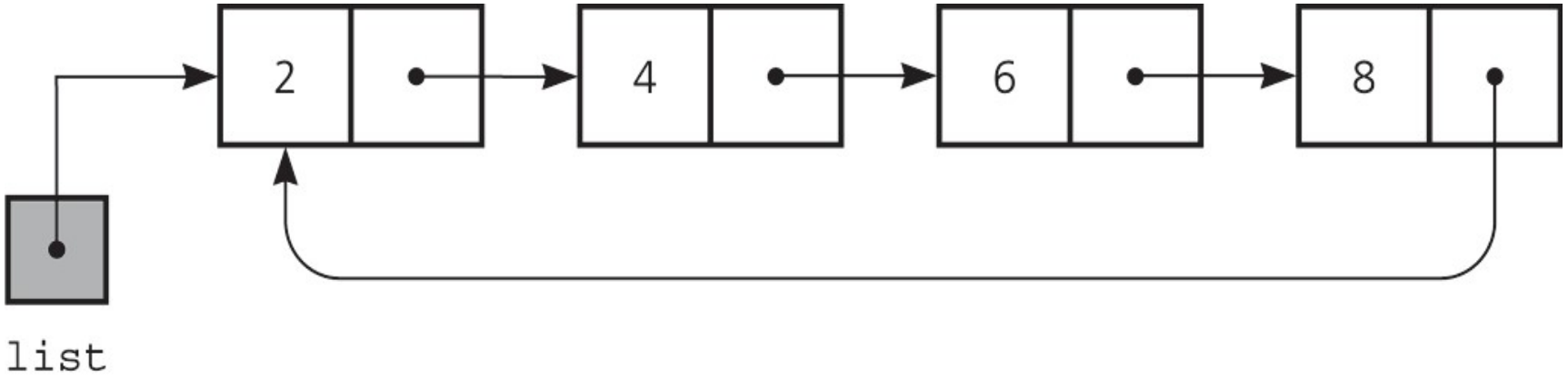
Doubly Linked Lists: Deletion




```
oldNode = current;
oldNode->prev->next = oldNode->next;
oldNode->next->prev = oldNode->prev;
free(oldNode);
current = head;
```

Circular Linked Lists

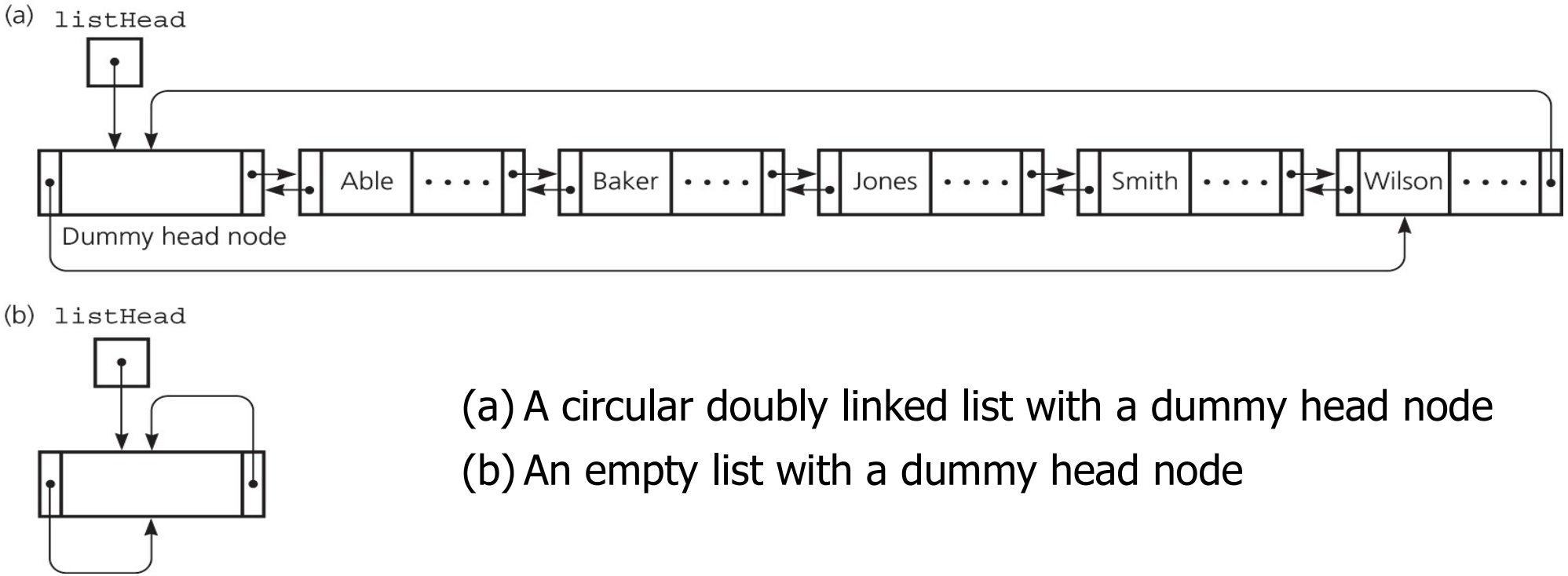
- Last node references the first node
- Every node has a successor
- No node in a circular linked list contains *NULL*



Circular Doubly Linked Lists

- Circular doubly linked list
  - **`prev`** pointer of the dummy head node points to the last node
  - **`next`** reference of the last node points to the dummy head node
  - No special cases for insertions and deletion

Circular Doubly Linked Lists



(a) A circular doubly linked list with a dummy head node

(b) An empty list with a dummy head node

Book Reference

- ✓ Fundamentals of Data Structures in C

  by E. Horowitz, S. Sahni, S. Anderson-Freed

- ✓ Data Structures using C and C++

  by Y. Langsam, M. Augenstein And A. M. Tenenbaum